

AI-Based Restoration of Degraded Inspection Images

Joint denoising and 2x super-resolution, built on a degradation model recovered from the data

SR. NO	ROLE	NAME	ACADEMIC YEAR
1	Team Leader	Aman Kumar Maurya	4th Year

Team: perceptron

Indian Institute of Technology Madras

8318354205 | 23f2001033@ds.study.iitm.ac.in

Problem Statement Addressed

AI-Based Restoration of Degraded Images for Semiconductor Inspection (KLA)

DESCRIPTION / DETAILS

- Inspection images arrive degraded by speckle noise, additive Gaussian noise and downsampling, applied in an undisclosed order
- Detail lost to degradation reduces the reliability of every downstream measurement and defect-detection step
- Input is half resolution and noisy; output must be clean at full ground-truth resolution
- The hidden test set includes out-of-distribution content, so the model must generalise beyond the training images

WHY IT MATTERS

- Semiconductor inspection is a precision-measurement task: invented detail is worse than no detail
- Cleaner images at higher resolution mean fewer missed defects and fewer false alarms
- Scored on restoration quality, end-to-end H100 throughput and reproducibility together

22.94 dB

doing nothing (bicubic upsample)

32.25 dB

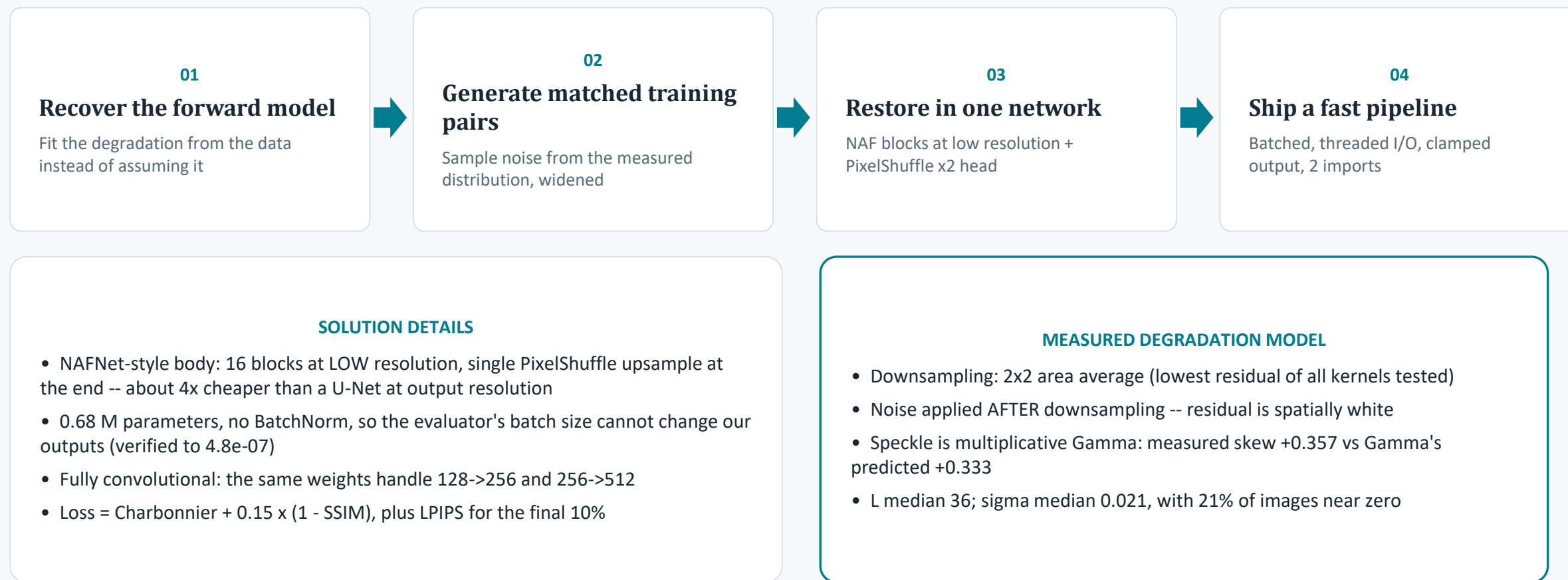
denoise-then-interpolate ceiling

3200 pairs

provided training data

Idea Description and Proposed Solution

One network, one forward pass: joint denoise + 2x super-resolution



Innovation and Uniqueness

What separates this from feeding pairs to a standard network

KEY INNOVATION -- we measured the degradation, we did not guess it

- Most entrants feed the provided pairs to a network. We first recovered the forward model by fitting the residual across all 3200 pairs
- The skew of the residual identifies the noise FAMILY: multiplicative Gaussian is symmetric, Gamma is not. Measured +0.357 against a predicted +0.333 pins it
- That let us synthesise unlimited training pairs matching the true distribution -- sampled from empirical quantile tables, then widened so the tails exceed what was observed
- A first version using assumed ranges was 50% noisier than reality and would have taught the model to over-smooth

+0.357 vs +0.333

measured vs predicted Gamma skew

0.68 M

parameters

COMPETITIVE ADVANTAGE

- Leakage-aware validation: samples are ordered by source image, so a random split leaks near-duplicates. We hold out 10 evenly-spaced contiguous blocks instead
- Every claim is reproducible by one command (python analyze_degradation.py)
- Deliberately no GAN: the brief asks for restoration without hallucinating structure, and invented detail is harmful in a measurement context
- Efficiency treated as a first-class deliverable, not an afterthought

1 command

regenerates the analysis

Impact and Benefits

Measured on a 320-image held-out split, never seen in training

Metric	Bicubic baseline	This solution	Improvement
PSNR	22.94 dB	27.54 dB	+4.60 dB
SSIM	0.5460	0.7594	+39%
LPIPS (lower better)	0.4478	0.3922	-12%

PRIMARY IMPACT

- Improves on ALL THREE scored axes at once -- perceptual metrics often move against PSNR, and here they do not
- Recovers detail lost to downsampling without inventing structure, which is what a measurement pipeline requires

QUANTIFIABLE OUTCOMES

- End-to-end inference: 46 images/second (21.7 ms each) on an NVIDIA T4
- 2.79 MB checkpoint -- deployable anywhere, no external downloads
- Same weights serve both 128->256 and 256->512 inputs

Honest limitation: training stopped at step 7500, short of convergence, so the model has not yet passed the denoise-then-interpolate ceiling of 32.25 dB. The cause was input-pipeline throughput, and it is the first thing we would fix.

Technology, Feasibility and Methodology

Everything below runs from a clean clone with no manual configuration

TECHNICAL STACK

- PyTorch 2.x, Python 3.11
- NAFNet-style CNN, 0.68 M params
- Trained on a single NVIDIA T4
- Inference: torch + numpy only
- No external datasets or pretrained restoration weights

IMPLEMENTATION

- `python run.py <in-dir> <out-dir>`
- Weights auto-loaded from models/
- Output dir created if absent
- Outputs clamped to [0,1], no NaN/Inf, verified
- Batched by resolution, threaded I/O, fp16 autocast

REPRODUCIBILITY

- Seeds fixed; config-driven, nothing hard-coded
- Every validation logs step, LR, all metrics, git hash and full config
- Model config embedded in the checkpoint
- Pre-flight gate blocks a broken run before it wastes GPU hours
- Verified from a fresh public clone

FEASIBILITY

Trained end-to-end within a single free Kaggle GPU session. The model is small enough to deploy on commodity hardware and fast enough for inline inspection, and the degradation engine can generate unlimited additional training data from any clean image set -- so the approach scales to new tools and processes without new paired acquisitions.

Repository, References and Disclosure

GITHUB REPOSITORY

<https://github.com/23f2001033/kla-restore>

Contains run.py, requirements.txt, README.md, models/best.pt, training and evaluation code, and the full results.

PROTOTYPE / SIMULATION VIDEO

Working solution demonstrated in the repository; inference reproducible with a single command.

RESEARCH BACKGROUND

- Chen et al., Simple Baselines for Image Restoration, ECCV 2022 -- NAFNet
- Wang et al., Real-ESRGAN, ICCV 2021 -- degradation modelling for real-world super-resolution
- Shi et al., Sub-Pixel Convolution, CVPR 2016 -- PixelShuffle upsampling
- Wang et al., Image Quality Assessment, IEEE TIP 2004 -- SSIM
- Zhang et al., The Unreasonable Effectiveness of Deep Features, CVPR 2018 -- LPIPS

EXTERNAL RESOURCE DISCLOSURE

- No external datasets used. No pretrained restoration weights used
- The model is trained from scratch on the official KLA data plus synthetic pairs derived from that same data
- Lpips (BSD-2-Clause) supplies the perceptual loss and metric; its VGG16 backbone carries ImageNet weights shipped with the package
- Training degrades gracefully without it, and run.py never imports it